DataStage – ETL TOOL

Requirements

- Minimum 5+ years' experience developing enterprise ETL solutions using IBM InfoSphere DataStage (mandatory).
- Strong expertise in ETL/ELT development, SQL scripting, performance tuning and enterprise data integration.
- Experience delivering complex data migration programmes within financial services, banking or insurance environments.
- Proven ability to automate deployments, troubleshoot production issues and collaborate effectively within Agile delivery teams.
- IBM Certified DataStage Developer certification or equivalent enterprise experience.

Nice To Haves

- Experience with Hogan and/or Fiserv Optis platforms.
- Knowledge of credit card, payments or core banking data models.
- French language proficiency (written and spoken) is considered an asset.
- Experience supporting large-scale platform or application migrations.
- Exposure to enterprise data warehousing and modern data engineering practices.

Responsibilities

- Design, develop and optimise IBM InfoSphere DataStage ETL/ELT jobs, reusable frameworks and SQL scripts to support enterprise data migration.
- Deliver scalable, high-performance data integration solutions that ensure data accuracy, integrity and operational excellence.
- Automate deployments, improve monitoring capabilities and troubleshoot ETL pipeline issues across complex enterprise environments.
- Collaborate with QA, business and technology teams to validate data, resolve defects and support successful programme delivery.
- Contribute to technical best practices, solution design and continuous improvement across the data engineering function.

For a recruiter/manager screening, these are the **most likely questions** based on this JD.

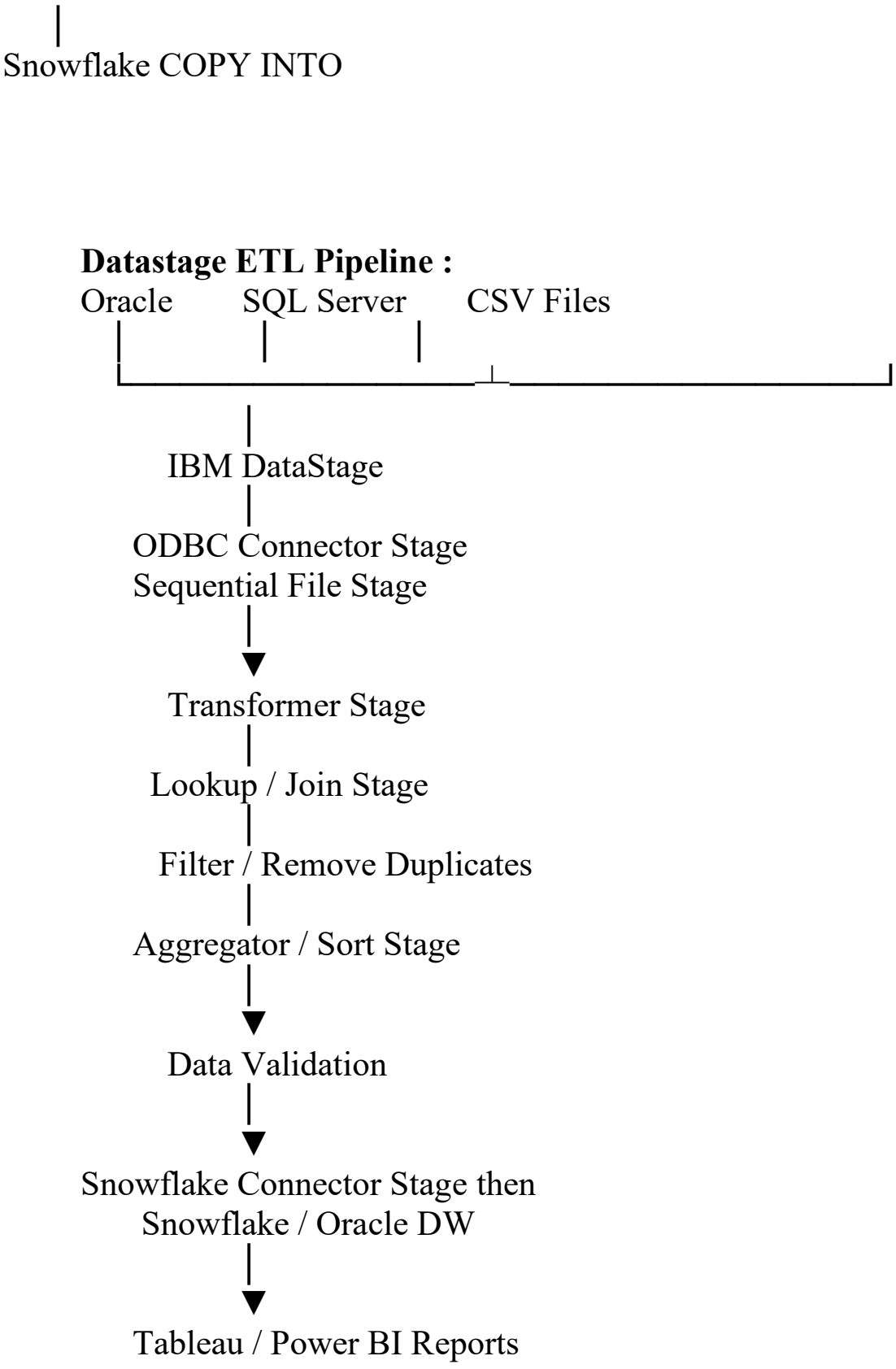
HR + Resume

1. Tell me about yourself.
   Hi, I'm Ankita Kshirsagar. I have around 4 years of experience as a Data Engineer, including about 3.5 years of hands-on experience with IBM DataStage, SQL, UNIX, and shell scripting for designing, developing, and maintaining enterprise ETL pipelines. I've worked extensively on building end-to-end ETL workflows to extract data from multiple source systems such as Oracle, SQL Server, and flat files, transform the data using DataStage Parallel Jobs, and load it into enterprise data warehouses while ensuring data quality, performance, and reliability.
   In my recent projects, I have also worked on cloud data engineering using AWS, Snowflake, Python, PySpark, and Databricks, where I was involved in modernizing legacy ETL pipelines and migrating workloads to cloud platforms. My responsibilities included analyzing

existing DataStage jobs, understanding transformation logic, optimizing SQL queries, writing shell scripts for job automation, and collaborating with business and technical teams to deliver scalable data solutions. I enjoy solving complex data integration challenges and am excited about opportunities involving ETL modernization, Snowflake, AWS, and enterprise data platforms.

2. Walk me through your Data Engineering experience.
   connection from oracle to DataStage
   Oracle -> **oracle connector stage** -> DataStage
   DataStage to AWS connection:
   1. DataStage writes a CSV or Parquet file to a local/staging directory.
   2. Use the **Amazon S3 Connector Stage** (if your DataStage version supports it).
   Oracle │ DataStage │ Amazon S3 Connector │ S3 Bucket

Complete pipeline:
Oracle
  │
Oracle Connector
  │
Transformer Stage
  │
Lookup / Join
  │
Sequential File Stage
  │
sales.csv
  │
Shell Script (AWS CLI)
  │
Amazon S3


Snowflake COPY INTO
Datastage ETL Pipeline :
Oracle
SQL Server
CSV Files
IBM DataStage
ODBC Connector Stage
Sequential File Stage
Transformer Stage
Lookup / Join Stage
Filter / Remove Duplicates
Aggregator / Sort Stage
Data Validation
Snowflake Connector Stage then
Snowflake / Oracle DW
Tableau / Power BI Reports

Answer: In our project, DataStage extracted data from Oracle and flat files, applied business transformations, and loaded the processed data into Snowflake. We primarily used the Snowflake Connector Stage for direct loads. For large batch loads, we first staged the files in Amazon S3 and then used Snowflake's COPY INTO command to load the data efficiently. This approach provided better performance and scalability for high-volume data.

3. Have you migrated DataStage jobs to Snowflake or AWS?
In one of my projects, I was involved in modernizing legacy IBM DataStage ETL jobs to a cloud-based architecture using Snowflake and AWS. We first analyzed the existing DataStage jobs, including Parallel Jobs, Transformer stages, Job Sequences, and XML metadata, to understand the extraction, transformation, and loading logic. We documented the data lineage, business rules, dependencies, and source-to-target mappings before starting the migration.
We then translated the transformation logic into modern ETL pipelines. Data was extracted from Oracle and flat files, landed in Amazon S3, processed using AWS Glue and PySpark, and loaded into Snowflake. We converted DataStage transformations such as lookups, joins, filters, aggregations, and surrogate key generation into SQL and PySpark. For large batch loads, we staged data in S3 and used Snowflake's COPY INTO command for efficient loading.

3. Are you comfortable working with legacy ETL systems?

Yes, absolutely. I have around 3.5 years of experience working with legacy IBM DataStage ETL systems. My responsibilities included developing and maintaining DataStage jobs, analyzing existing ETL

workflows, understanding transformation logic, troubleshooting production issues, optimizing performance, and supporting modernization initiatives. I'm comfortable working with legacy environments while also helping migrate them to modern platforms like Snowflake and AWS.

IBM DataStage

11. What is IBM DataStage?

IBM DataStage is an enterprise ETL tool from IBM InfoSphere used to design, develop, schedule, and manage data integration workflows. It extracts data from multiple sources such as Oracle, SQL Server, flat files, and APIs, applies business transformations, and loads the processed data into target systems like data warehouses or cloud platforms.
It provides a graphical development environment with reusable stages such as Transformer, Join, Lookup, Aggregator, and Sequential File, making it easy to build scalable ETL pipelines. It supports both parallel processing for high-performance data movement and job sequencing for workflow orchestration, making it widely used in enterprise data warehousing projects.

12. Explain the DataStage architecture.

Data Stage Tools:

Designer – Build ETL jobs.
Director – Run and monitor jobs.
Administrator – Configure projects and users.
Manager – Manage metadata and repository objects (mainly in older versions).

Tool	Purpose
Designer	Develop ETL jobs (Parallel Jobs, Server Jobs, Job Sequences)
Director	Run, schedule, monitor, and troubleshoot jobs
Administrator	Create projects, configure users, permissions, and environment variables
Manager *(older versions)*	Manage metadata, repository objects, imports/exports, table definitions
Repository	Stores metadata, job designs, parameters, and table definitions (backend component, not a GUI tool)
Engine	Executes DataStage jobs (backend execution engine)

13. What are Server Jobs and Parallel Jobs?

Differences

Server Jobs	Parallel Jobs
Sequential processing	Parallel processing
Single node	Multiple nodes/CPUs
Suitable for small data	Suitable for large datasets (GBs/TBs)
Lower performance	High performance and scalability
Older architecture	Modern enterprise architecture
Limited partitioning	Supports data partitioning and parallel execution

Which stages have you used?

- **Oracle Connector Stage** – Extract data from Oracle databases.
- **ODBC Connector Stage** – Connect to SQL Server and other relational databases.
- **Sequential File Stage** – Read from and write to CSV and flat files.
- **Transformer Stage** – Apply business rules, data type conversions, derivations, and null handling.
- **Lookup Stage** – Retrieve reference data from lookup tables.
- **Join Stage** – Join data from multiple source datasets.
- **Merge Stage** – Merge sorted datasets efficiently.
- **Filter Stage** – Filter records based on business conditions.
- **Sort Stage** – Sort data before joins or loading.
- **Aggregator Stage** – Perform SUM, COUNT, AVG, MIN, and MAX operations.
- **Copy Stage** – Copy datasets without transformation.
- **Modify Stage** – Change metadata and data types for better performance.
- **Remove Duplicates Stage** – Eliminate duplicate records.

- **Funnel Stage** – Combine multiple datasets into a single output.
- **Surrogate Key Generator Stage** – Generate surrogate keys for dimension tables.
- **Peek Stage** – Debug and inspect data during development.
- **Dataset Stage** – Store intermediate datasets for better performance

11. What is a Transformer Stage?

The **Transformer Stage** is the most commonly used transformation stage in IBM DataStage. It is used to apply business logic and transform data before loading it into the target system. Using the Transformer Stage, we can:

Convert data types

Handle null values

Create derived columns

Apply IF-THEN-ELSE conditions

Perform calculations and string manipulations

Map source columns to target columns

Generate stage variables for intermediate calculations

Route records to different output links based on conditions

12. Difference between Lookup and Join.

Normal Lookup = = exact match
Range Lookup = <, >, <=, >= range match.

Lookup Stage	Join Stage
Used to retrieve matching records from a **small reference dataset**	Used to combine **two or more large datasets**
One input is the main dataset, the other is a lookup table	All input datasets are treated equally
Faster for small lookup tables	Better for large datasets
Lookup dataset is loaded into memory	Does not require loading the entire dataset into memory
Returns matching data from the lookup table	Supports Inner, Left Outer, Right Outer, and Full Outer joins
Best for dimension/master data lookups	Best for combining transaction tables

13. Difference between Join and Merge.

Join Stage	Merge Stage
Combines two or more datasets based on a key	Merges two or more **already sorted** datasets
Input data **does not have to be sorted**	**Input data must be sorted** on the join key
Performs matching during execution	Sequentially merges sorted records
More memory intensive	More efficient and faster for large sorted datasets
Supports Inner, Left, Right, and Full Outer joins	Mainly used to merge sorted datasets (Inner, Left, Full Outer supported)
Used when data is unsorted	Used when data is already sorted

14. What is a Sequential File Stage?

The **Sequential File Stage** is a source and target stage in IBM DataStage used to **read data from** or **write data to** sequential files

such as CSV, TXT, fixed-width, or delimited files.

15. What is an ODBC Connector?

16. What are Hashed Files?

A **Hashed File** in IBM DataStage is a special type of file used to **store and retrieve data quickly using a key**. Instead of scanning every record, DataStage uses a hash algorithm to locate the required record directly, making lookups much ster.

Create hashed file: Sequential File → Transformer → Hashed File Stage ↓ Customer_Hash

17. What is a Job Sequence?

A **Job Sequence** in IBM DataStage is used to **orchestrate multiple ETL jobs**. Instead of manually running each job one by one, a Job Sequence executes them in a defined order, handles dependencies, passes parameters, performs error handling, and manages workflow execution.

18. What are Parameters in DataStage?

Parameters in IBM DataStage are **variables** used to make ETL jobs reusable and flexible. Instead of hardcoding values such as file paths, database names, usernames, passwords, or dates, we define them as parameters and pass the values at runtime.

Common Parameters

Database name

Username & Password

File path

Source/Target table name

Processing date

Schema name

EX: sales_job.par

DB_HOST=server01

DB_USER=dsuser

INPUT_FILE=/data/sales.csv

RUN_DATE=2026-08-10

We commonly use .par for external parameter files, but the extension itself isn't mandatory; what matters is how the script/job reads and passes the parameter values

19. How do you improve DataStage performance?

I improve DataStage performance using several best practices:
Use Parallel Jobs instead of Server Jobs for large datasets.

Partition data to process records in parallel across multiple nodes.

Use the Modify Stage instead of the Transformer Stage for simple data type conversions, as it consumes fewer resources.

Use the Merge Stage instead of the Join Stage when the input datasets are already sorted.

Use Hashed Files or Lookup Stages for small reference tables to reduce repeated database queries.

Filter unnecessary records and columns early in the pipeline to minimize the amount of data being processed.

Optimize SQL queries by pushing filters and joins to the database whenever possible.

Avoid unnecessary Sort stages, since sorting is resource-intensive.

Tune partitioning methods such as Hash, Round Robin, or Entire Partitioning based on the data and workload.

Monitor job logs and identify bottlenecks using DataStage Director.

Shell Scripting

46. Write a shell script to move files.
47. How do you pass parameters?
48. What are exit codes?
49. Difference between $* and $@.
50. How do you schedule shell scripts?

XML

51. What is XML metadata in DataStage?

    XML metadata in IBM DataStage is the XML representation of a DataStage job. It contains information about the job design, including stages, links, columns, parameters, transformations, and job properties. It is commonly used to **export, import, migrate, and analyze DataStage jobs** between environments.

52. Have you parsed XML files?

Yes. I have worked with XML files in DataStage, primarily to understand job metadata and ETL logic during migration and modernization projects. I parsed XML files to extract information such as stages, source and target mappings, transformation logic, parameters, and job dependencies. This helped us document the existing ETL process and accurately recreate the logic in modern platforms like Snowflake and AWS.

53. How do you extract XML elements?

XML elements are typically extracted using **XPath expressions**. In IBM DataStage, I use the **XML Input Stage** to read the XML file, define the XML schema (XSD if available), and map the required XML elements to DataStage columns using XPath. XPath (XML Path Language) is a query language used to navigate and extract specific elements or attributes from an XML document. It works similarly to how SQL is used to query relational databases

---

ETL Modernization

56. What challenges did you face during migration?

The biggest challenges were understanding undocumented legacy ETL logic, converting complex DataStage transformations into SQL, maintaining data accuracy, optimizing performance, handling job dependencies, and validating the migrated data. We resolved these through detailed job analysis, XML metadata review, source-to-target reconciliation, and extensive testing before production

57. How do you compare source and target data?
To compare source and target data, I perform row count validation, compare aggregate values such as SUM and COUNT, validate key business fields, check for duplicates and null values, and perform source-to-target reconciliation using primary keys. I also validate business rules and compare sample records to ensure the migrated pipeline produces the same results as the legacy system before moving it to production."

AI & Automation

91. Have you used ChatGPT, GitHub Copilot, or AI coding tools?

92. How can AI help modernize DataStage jobs?
AI can significantly accelerate DataStage modernization by analyzing legacy ETL jobs and automatically identifying transformation logic, source-to-target mappings, SQL queries, and job dependencies. Instead of manually reviewing hundreds of DataStage jobs, AI tools can help generate documentation, explain complex business rules, and even suggest equivalent SQL or PySpark code for modern platforms like Snowflake or AWS Glue."

93. How would you convert XML metadata into SQL using AI?

94. Have you automated ETL documentation?

95. How do you validate AI-generated code?

Behavioral

96. Describe a production issue you resolved.

97. Describe a failed deployment.

98. How do you prioritize tasks?

99. Tell me about a challenging migration project.

100. Why should we hire you?

Why capco?
Capco stands out because of its strong focus on digital transformation and data-driven solutions in the financial services industry. I'm excited by the opportunity to work on large-scale data engineering and cloud modernization projects using technologies like DataStage, Snowflake, AWS, and SQL. I also appreciate Capco's collaborative culture, emphasis on innovation, and investment in employee growth. I believe this role aligns well with my experience in ETL development and modernization, and it would allow me to contribute while continuing to grow as a Data Engineer.

Project In Details:
Oracle Source
↓
Oracle Connector
↓
DataStage Parallel Job
↓
Transformer Stage
↓
Lookup / Join Stage
↓
Sequential File Stage
↓
sales.csv
↓
Unix Shell Script + AWS CLI
↓

Amazon S3
↓
Snowflake Stage
↓
COPY INTO
↓
Snowflake Target Table

Q: How did DataStage connect to Oracle?

Using the **Oracle Connector stage**.

Typical configuration:

- Oracle hostname
- Port
- Service name/SID
- Username/password
- SQL/query or table
- Schema/table metadata

Q: How did DataStage connect to Oracle?
Using the **Oracle Connector stage**.
Typical configuration:
- Oracle hostname
- Port
- Service name/SID
- Username/password
- SQL/query or table
- Schema/table metadata

Q: How did you manage passwords?
"Credentials were secured through environment/project-level configuration rather than hardcoded inside the job."

Q: Full load or incremental load?

For initial migration:
Oracle → Full extraction → Snowflake

For daily processing:
Oracle
↓
WHERE LAST_UPDATED_TS > previous_watermark
↓
DataStage
↓
Snowflake
Use a **watermark/control table** to track the last successful extraction timestamp.

Q: What did you do inside Transformer?
Main tasks:

- Data type conversion
- NULL handling
- Business rules
- String manipulation
- Date conversion
- Derived columns
- Filtering
- Validation
- Reject routing

Ex:
Derived column:
TOTAL_AMOUNT = QUANTITY * UNIT_PRICE
NULL handling conceptually:
If IsNull(CUSTOMER_ID)
Then -1
Else CUSTOMER_ID

Q: Why Transformer instead of doing everything in SQL?

“Simple filters and aggregations can be pushed down to Oracle, while DataStage Transformer is useful for ETL-specific business rules, validations, derivations, and reject handling.”

Q: Why Lookup?
“To enrich incoming transaction records with reference information.”

Q: What happens if lookup doesn't find a record?
Depending on business requirements:
Reject record
OR
Assign default value
OR
Continue with NULL/default key
Example:
Unknown customer → CUSTOMER_KEY = -1

Parallelism :
One stage done need to wait for another to execute – another start parallely .
In **IBM DataStage Parallel Jobs**, there are mainly **3 types of parallelism**:

1. **Pipeline Parallelism** — Different stages process data simultaneously.
   Example: while Oracle Connector reads the next records, Transformer processes previous records, and the output stage writes already-transformed records.
   Oracle → Transformer → Sequential File all work concurrently.
2. **Partition Parallelism** — Data is divided into multiple partitions and the **same operation runs simultaneously** on each partition.
   Example: 100M rows can be distributed across 4 nodes, with each node processing a portion of the data. Common partitioning methods include **Hash, Round Robin, Random, Range, Same, and Entire**.

3. **Component/Job Parallelism** — Independent stages, branches, or jobs can execute simultaneously.
   Example, after extraction, independent processing branches can run at the same time:

Partition Types :

Partition	What it does	Common use
Hash	Same key values go to the same partition	Join, Aggregator, Remove Duplicates
Round Robin	Rows distributed evenly across partitions	Balance data/load
Random	Rows randomly distributed	General load distribution
Entire	Every partition receives the **entire dataset**	Small reference/lookup data
Same	Keeps the existing partitioning	Avoid unnecessary repartitioning
Range	Distributes based on value ranges	Range-based processing/sorting
Modulus	Uses modulus on an integer key	Numeric-key distribution
DB2	Uses DB2-compatible partitioning	DB2 integration

1. Hash Partitioning
Same key → same partition.
Customer_ID

101 → Node 1
102 → Node 2
101 → Node 1
103 → Node 3

2. Round Robin:
DataStage distributes records one by one cyclically across the available partitions.

Q: Which partitioning would you use for joining Sales and Customer on CUSTOMER_ID?
Answer:
"Usually **Hash partitioning on CUSTOMER_ID for both inputs**, so records with the same customer ID are processed in the same partition."

Which partition need to use for lookup stage?
For a **Lookup Stage**, partitioning depends on the size of the reference data:

- **Small reference dataset → Entire partitioning** is commonly used. Each processing node gets a complete copy of the reference data, so every incoming row can perform the lookup locally.
- **Large reference dataset → Hash partitioning** on the lookup key can be more appropriate, with both input and reference data partitioned consistently on the same key.

Q.How to get non matching record from join ?

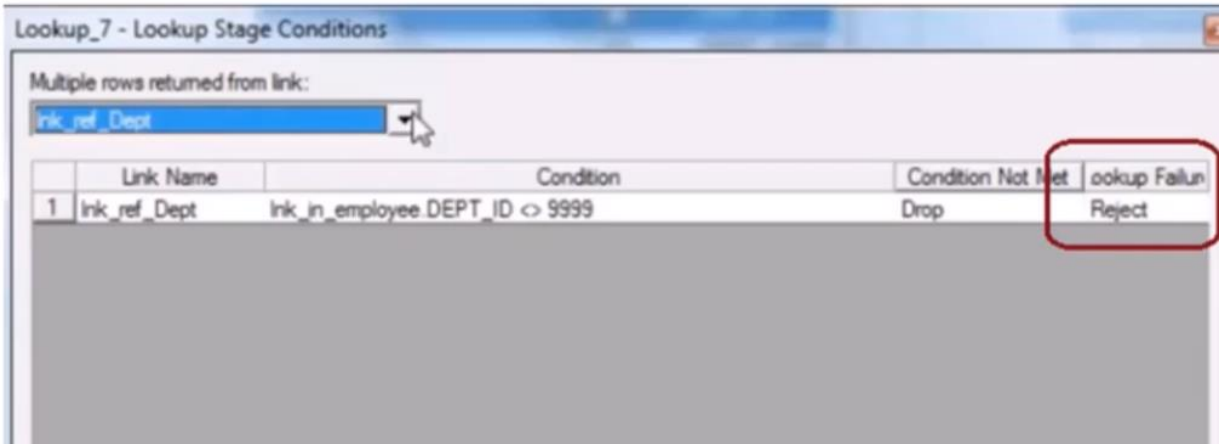
The Join Stage does not provide a separate reject link specifically for unmatched rows. I use a **Left/Right/Full Outer Join** depending on

which unmatched records I need, then use a **Transformer Stage to identify NULL values from the non-matching side and route those records to a separate output**.
Outer Join → Transformer → NULL check → Non-matching records.

Q . Suppose one employee assigned to 2 project but lookup taking only one time that employee then how to handle?

Lookup option :
Multiple Rows Returned from Link = True

Q Use lookup as left join ?
Normally lookup is used as inner join but below option set to continue will work as left join for look up
Lookup_Failure = Continue



Multiple Rows Returned From Link = False → one matching row
Multiple Rows Returned From Link = True → all matching rows

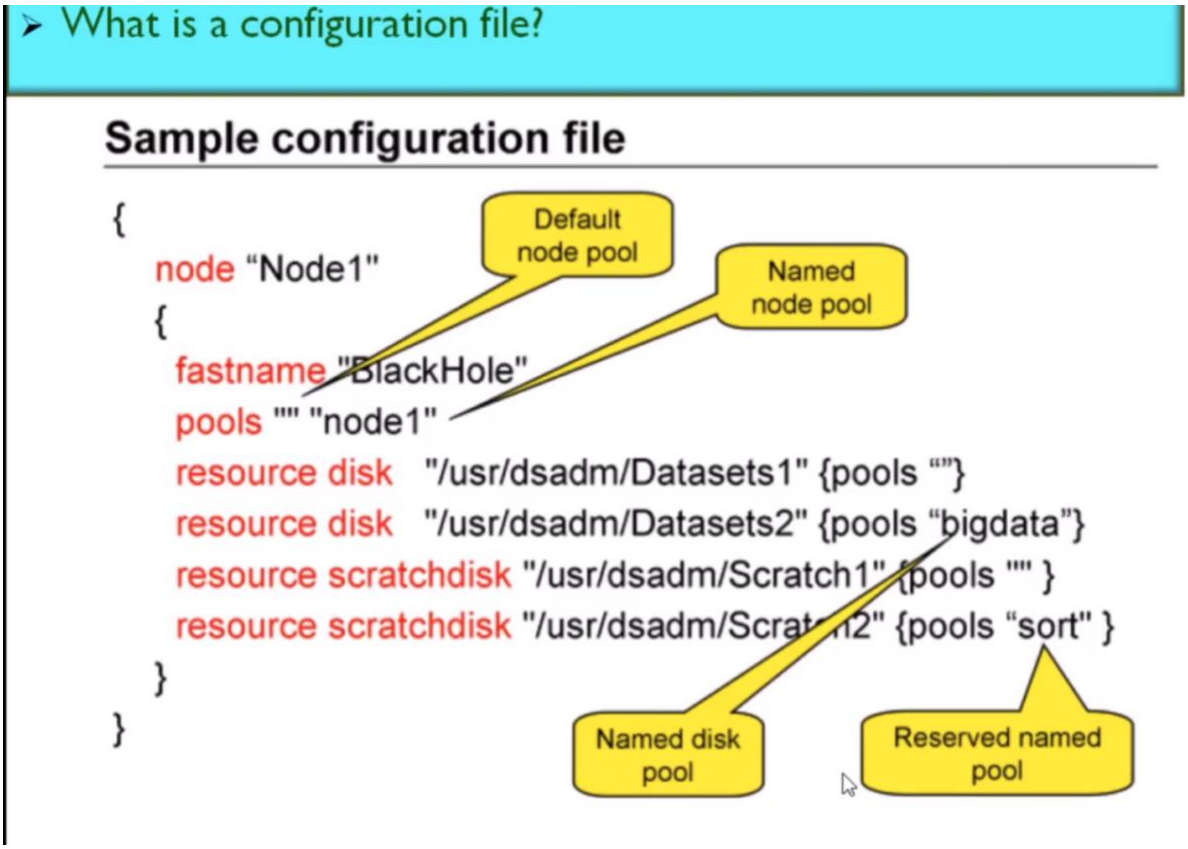
Q . Normal v/s Sparse Lookup

Normal Lookup :-

-Reference data needs to be in memory
-Might provide poor performance if the reference data is huge as it has to put all the data in memory.
-Normal Lookup can have more than one reference link.
-Normal lookup can be used with any database

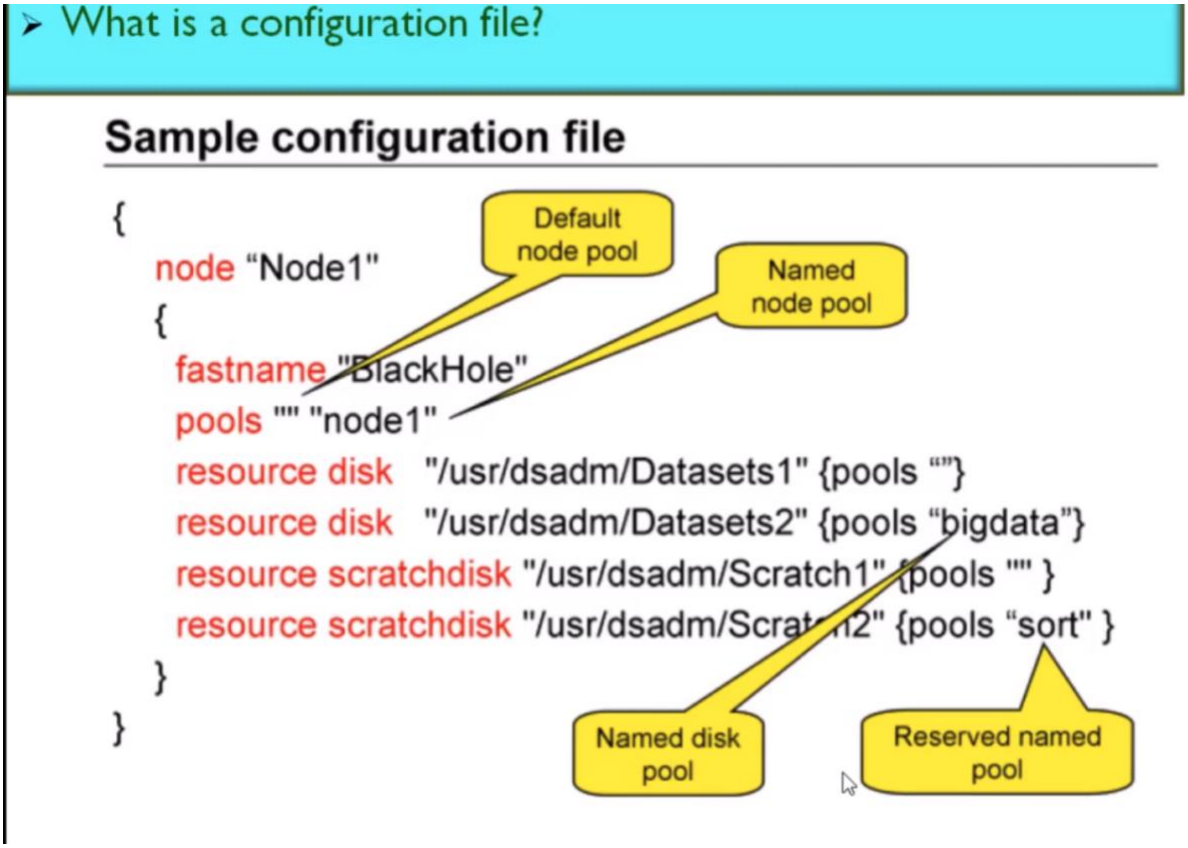
Sparse Lookup :-

-Sparse Lookup directly hits the database.
-If the input stream data is less and reference data is more like 1:100 or more in such cases sparse lookup is better.
-It can have only one reference link.
It can be used with databases.
-Sparse lookup sends individual sql statements for every incoming row.



You can define number of nodes or can define parallelism degree

Can make multiple config file for multiple stages
Apt.congif



Q . How to delete dataset?

- Descriptor file and Data file.
- Virtual Data Sets exist in-memory correspond to DataStage Designer links.
- Persistent Data Sets are stored on-disk.

- Data Set Management utility
- Orchadmin delete demo.ds

➢ How can you make the Sequential File Stage run in parallel?

Parallelism in Sequential files:

➢ **Read from Multiple Nodes - Yes**

➢ **Number Of readers per node** – more than one.

Number Of readers per node - Specifies the number of instances of the file read operator on a processing node. The default is one operator per node per input data file.
Read from multiple nodes - Set this to Yes to allow individual files to be read by several nodes. This can improve performance on a cluster system.

Can set only one property from both **not two**

Q .Run time schema reading
RCP

➢ How can you define dynamic column definitions in a Sequential File?

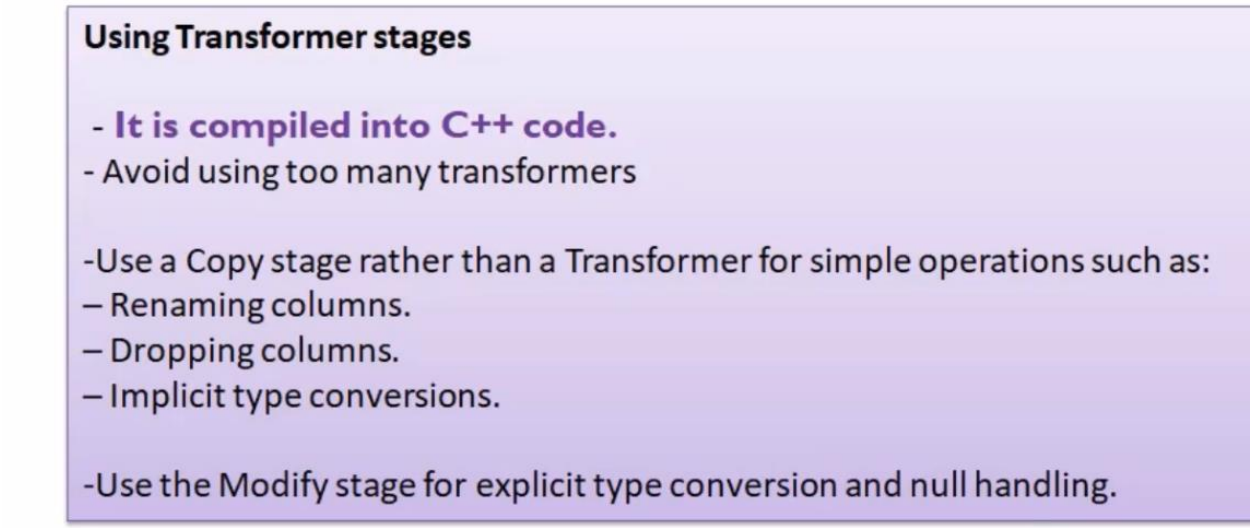
Runtime column propagation (RCP)

-Propagates extra columns through the job when schema is defined partially

-Can be enables at a project level through Datastage Administrator client.

- Can be set at a stage level on Output tab of the stage.

➢ Use of Transformer Stage



We can avoid using transformer stage cause its heavy performance.

Q: Source columns are all String. How do you validate that Date of Birth is a valid Date and reject invalid records?

Use a **Transformer Stage** with IsValid().

Job design

Sequential File
↓
Transformer
/ \
Valid Reject
↓ ↓
Target Reject File

In the Transformer, validate the DOB string against the expected date format.

For example, if DOB comes as YYYY-MM-DD:

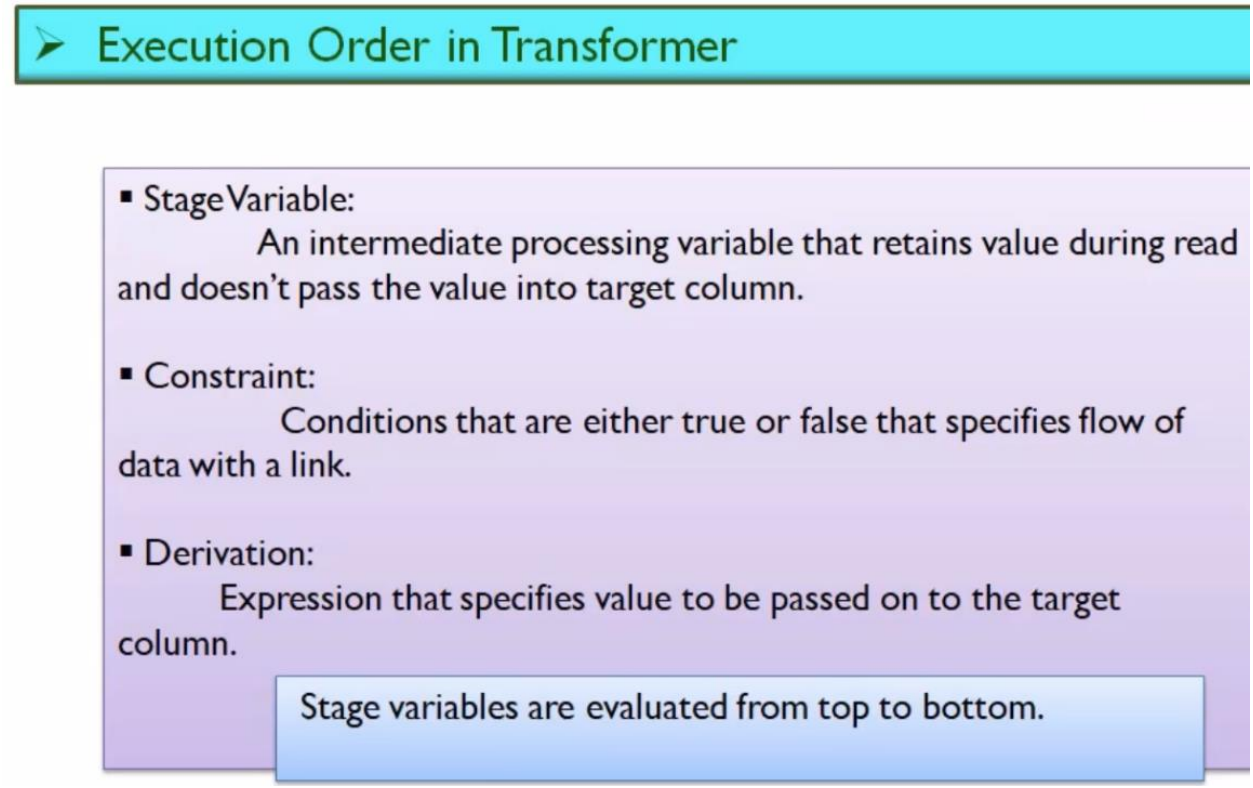
IsValid("date", StringToDate(DOB, "%yyyy-%mm-%dd"))

You can apply validations for multiple/all columns in a single Transformer Stage in DataStage.

Q. string function in transformer :

Function	**Purpose**	**Example**
Trim()	Remove leading/trailing spaces	Trim(Name)
UpCase()	Convert to uppercase	UpCase(Name)
DownCase()	Convert to lowercase	DownCase(Name)
Len()	Find string length	Len(Name)
Left()	Get characters from left	Left(Name,3)
Right()	Get characters from right	Right(Name,4)
Field()	Extract part using delimiter	Field(Name,",",1)
Convert()	Replace/convert characters	Convert("-", "/", DateStr)
Change()	Replace substring	Change(Name,"ABC","XYZ")
Index()	Find position/occurrence of substring	Index(Name,"A",1)
Count()	Count occurrences	Count(Name,",")
Space()	Generate spaces	Space(5)
Str()	Repeat a string	Str("*",5)
Fmt()	Format a string/value	Formatting output

Q . In Transformer stage variable evaluate first or constraint first?
Stage Variable → Constraint → Output derivation.
In a Transformer Stage, **stage variables are evaluated before output constraints**. Stage variables are evaluated in their defined order, and then constraints determine which output link receives the record.



Q . Handle Null values .
Ex:
If IsNull(Input.Salary)
Then 0
Else Input.Salary
Functions:
I generally handle NULL values in the **Transformer Stage** using functions such as IsNull(), NullToValue(), and SetNull(), depending on the business requirement.

Q .Exmaple

Design a job to implement the below requirement –

Input Data

Month	Sales	Prod
01	45	AA
02	30	AA
.	.	.
.	.	.
12	21	AA

Output Data (YTD)

Month	Sales	Prod
01	45	AA
02	75	.
.	.	.
.	.	.
12 (Sum of all 12 months)		

➢ Stage Variables

stgSUM = IF Input.Prod = stgProd
THEN stgPrevMonth Sales+ Input.Sales
ELSE Input.Sales

stgPrevMonthSales = Input.Sales

stgProd = Input.Prod

Sort and Hash Partition on Product ID
Order by Product ID and Month

Q . Job Design (send output to different links):

Design a job –

If Active Flag = T and Year > 2010 then output to First link
If Active Flag = T and Year = 2010 then output to Second link
If Active Flag = F and Year = 2012 then output to Third link
All other records to be collected in a separate output link.

I would use one Transformer Stage with four output links. I would define constraints on the first three links according to the Active Flag and Year conditions and configure the fourth link as **Otherwise** to capture all remaining records.

Q .Datastage Macros :
In **IBM DataStage**, macros are predefined variables that provide information about the **current job, project, invocation, user, date/time, etc.** They are commonly used in job parameters, audit tables, filenames, and logging.

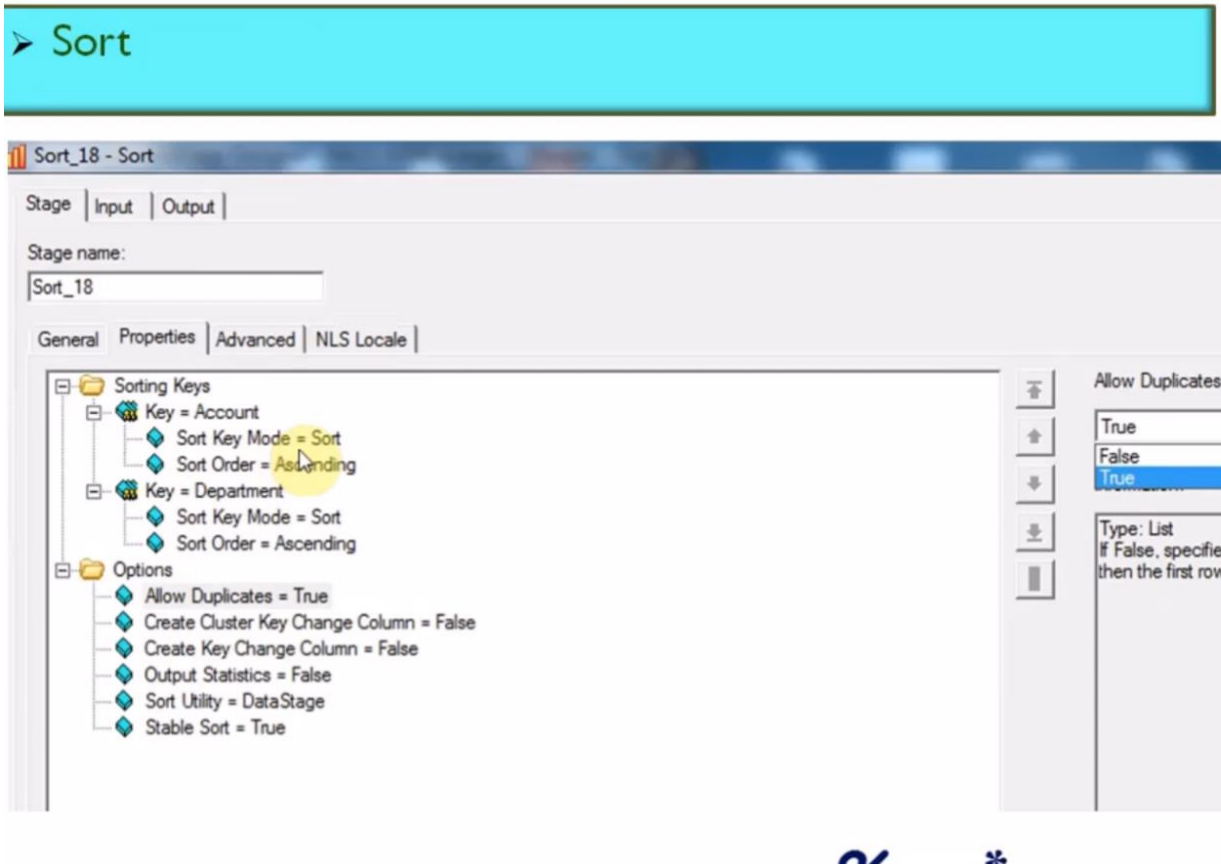
Macro	**Meaning**
DSJobName	Current job name
DSJobStartDate	Job start date
DSJobStartTime	Job start time

Macro	Meaning
DSJobStartTimestamp	Job start timestamp
DSProjectName	Current project name
DSHostName	Host/server name
DSUserName	User running the job
DSJobInvocationId	Invocation ID for multi-instance jobs

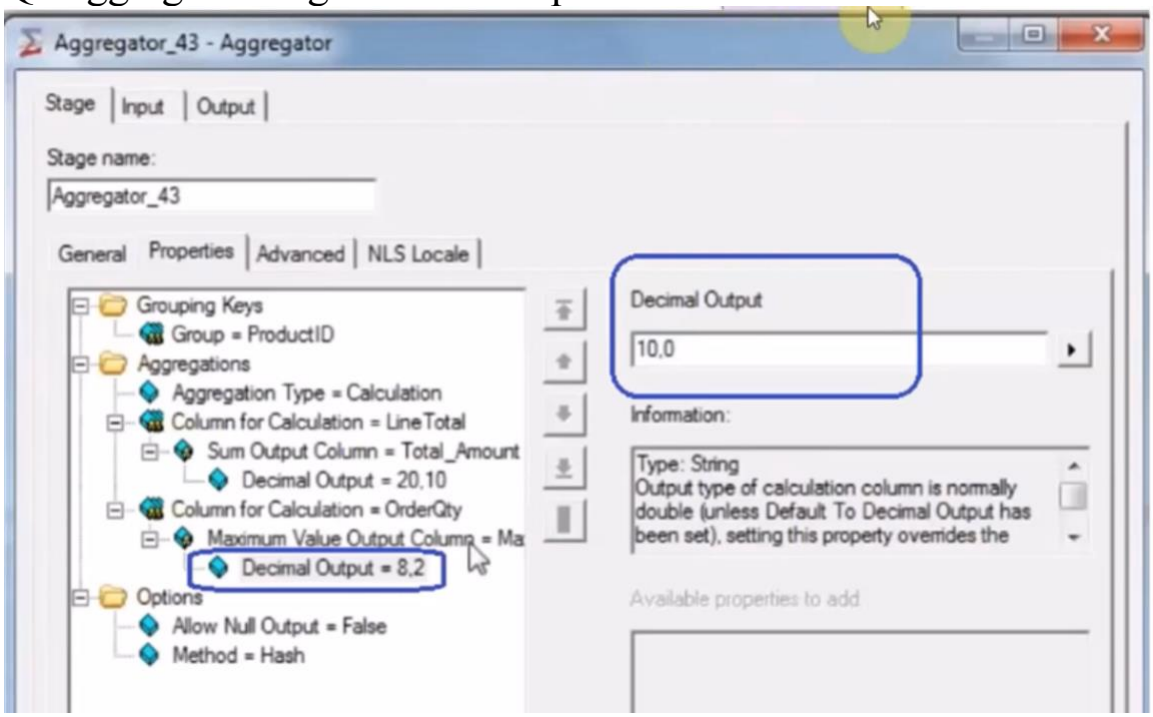
Q. Remove duplicate

1. Transformer
2. Remove Duplicate stage (Duplicate to retain : option First / Last)
3. Sort stage

Sort : allow duplicate True/False



Q. Aggregator stage – Default op: Decimal



Method : Hash and Sort (choose one for grouping hash - Default
)

Aggregator - Hash

- Limited number of distinct groups.
- Calculations are made for all groups and stored in memory (Has table structure and
- hence the name).
- Incoming data does not need to be pre-sorted.
- Results are output after all rows have been read.
- Useful when the number of unique groups is small.

Aggregator - Sort

- Large (or unknown) number of distinct key column values
- Requires inputs pre-sorted on key
- Results are output after each group
- Can handle unlimited number of groups

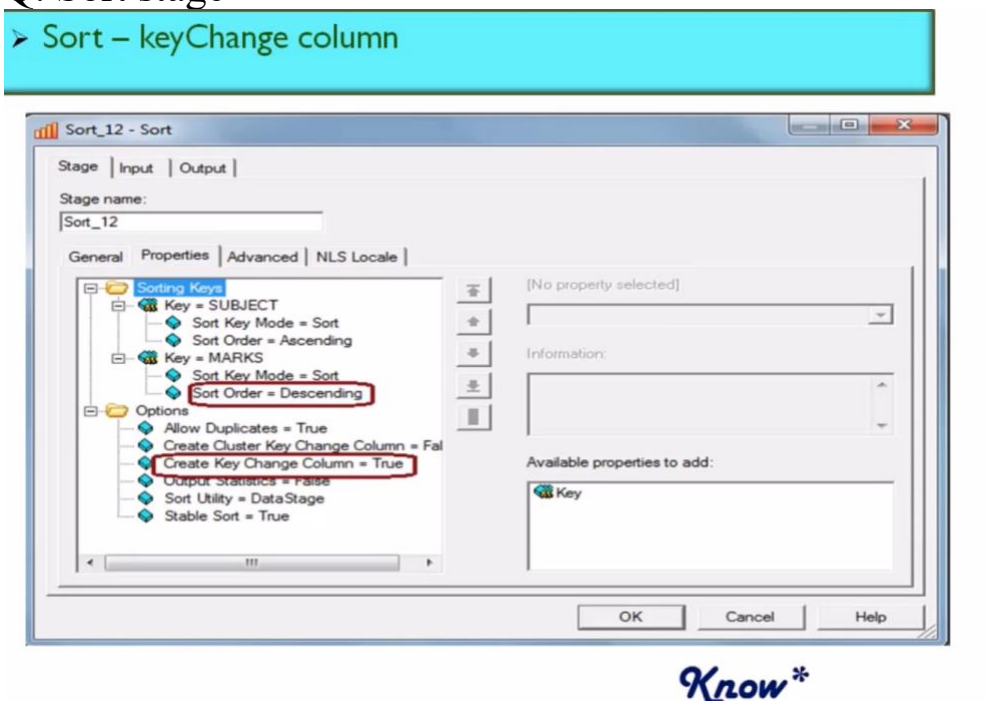
Aggregation Types:

1. Calculation
2. Re-calculation
3. Count-Row

Aggregator Operations :

Operation	Purpose
Sum	Total value
Count	Number of records
Min	Minimum value
Max	Maximum value
Mean	Average
First	First value in group
Last	Last value in group
Standard deviation / variance	Statistical calculations

Q. Sort stage



Know*

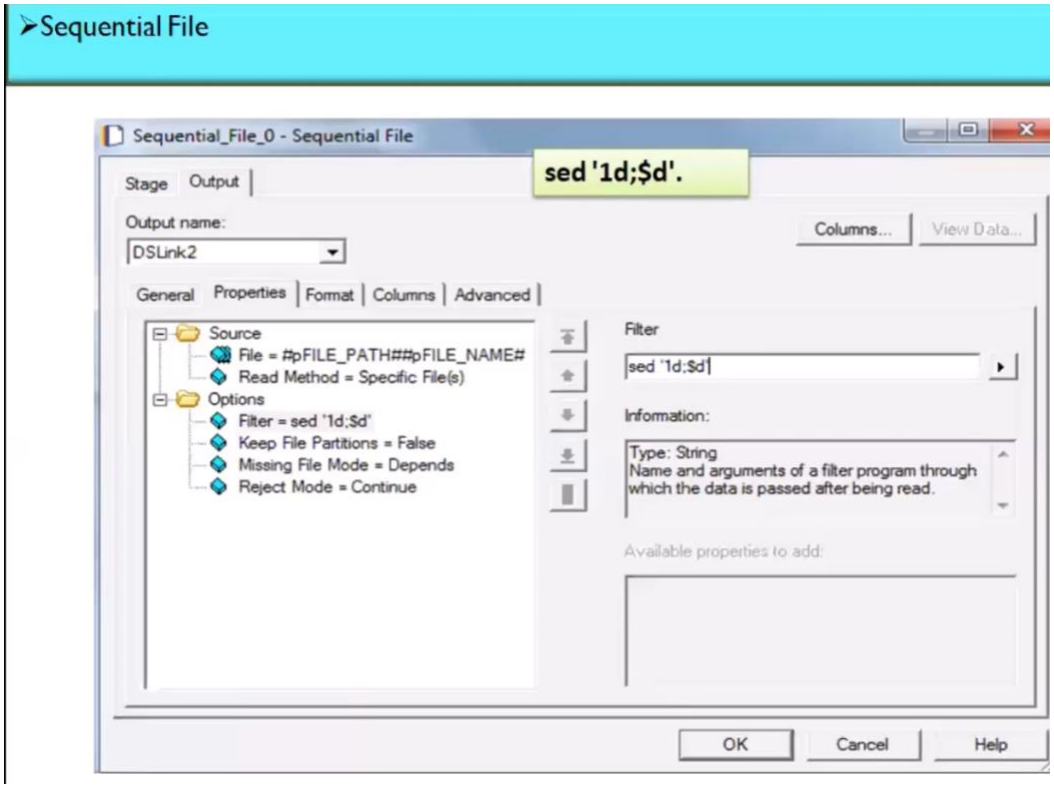
Option	What it does	Example/use
Sort Key	Column(s) used for sorting	Customer_ID, Date
Sort Order	Ascending or Descending	Date DESC
Case Sensitive	Controls case-sensitive comparison	ABC vs abc
Nulls Position	Controls where NULLs appear	NULL first/last
Allow Duplicates	Keeps/removes duplicate key records	False removes duplicates
Stable Sort	Preserves original order for equal-key records	Same Customer_ID maintains input order
Unique	Ensures unique sort-key records	Deduplication
Create Key Change Column	Adds indicator when sort key changes	1 new group, 0 same group
Restrict Memory	Limits memory available for sorting	Large-volume processing
Output Statistics	Produces sort-related statistics	Monitoring/tuning

Create Key Change Column :
Example :

Prod	Month	Sales	KeyChange
AA	01	45	1
AA	02	30	0
AA	03	20	0
BB	01	50	1
BB	02	25	0

Q . Remove header and trailer of file

Sed '$1d;$d'.



Q . I want the header trailer and data is single file
Use Funnel Stage and choose Funnel Type :Sequence

➢ Funnel

Funnel Stage

It copies multiple input data sets to a single output data set.

Funnel Stage - Modes

Continuous Funnel
It combines the records of the input data in no guaranteed order.

Sort Funnel
It combines the input records in the order defined by the value(s) of one or more key columns and the order of the output records is determined by these sorting keys.

Sequence
It copies all records from the first input data set to the output data set.

Another way using unix command:

DataStage flow

```
Generate Header ──→ header.txt
Generate Data   ──→ data.txt
Generate Trailer──→ trailer.txt
                          ↓
                  Execute Command
                          ↓
cat header.txt data.txt trailer.txt > final.txt
                          ↓
                     final.txt
```

DataStage Performance Tuning — Key Points

1. **Push processing to source**
   - Filter data in Oracle/DB rather than extracting everything.
   - Select only required columns.
   - Use indexed columns in predicates where appropriate.
   - Avoid SELECT *.
2. **Use proper partitioning**
   - **Hash** → Join, Aggregator, Remove Duplicates by key.
   - **Round Robin** → Even distribution.
   - **Same** → Avoid unnecessary repartitioning.
   - Watch for **data skew**.
3. **Avoid unnecessary Sort stages**
   - Sorting is expensive: CPU + memory + disk.
   - Don't sort again if upstream data already satisfies required ordering.
4. **Lookup optimization**
   - Small reference → Normal Lookup.
   - Huge reference but few required rows → Sparse Lookup.
   - Two huge datasets → Prefer Join.
   - Avoid loading massive reference data unnecessarily.
5. **Join optimization**
   - Partition both inputs on the **same join key**.
   - Sort inputs as required.
   - Filter unnecessary records before Join.
   - Remove unused columns before Join.
6. **Aggregator optimization**
   - Hash partition on grouping keys.
   - Use appropriate Hash/Sort aggregation method.
   - Aggregate early when it significantly reduces downstream volume.
7. **Transformer optimization**
   - Avoid unnecessary Transformer stages.
   - Combine related simple transformations where maintainable.
   - Use stage variables for repeated expressions.

- Filter/reject unwanted records early.

8. **Use DataSet for intermediate data**
   Instead of:
   Job1 → Sequential File → Job2
   for DataStage-internal intermediate processing, prefer when appropriate:
   Job1 → DataSet → Job2
   DataSet supports parallel processing and preserves DataStage metadata/partitioning information.
9. **Sequential File optimization**
   - Avoid creating one enormous sequential file when parallel downstream processing is possible.
   - Use multiple appropriately sized files where required.
   - Avoid unnecessary intermediate disk I/O.
10. **Node configuration**

- DataStage parallel jobs use processing nodes.
- More nodes can improve performance **only when the workload can benefit from additional parallelism**.
- More nodes do not automatically mean faster jobs.

11. Split job if one stage has heavy logic (Modular Design)

Q. Run datastage using UNIX
Connect to server first :
ssh username@datastage-server

1. **Run DataStage jobs from Unix**
   dsjob -run ProjectName JobName

   Run and wait for completion:
   dsjob -run -wait ProjectName JobName

   Parameters:

dsjob -run -wait \ -param InputFile=/data/sales.csv \ -param RunDate=2026-08-09 \ ProjectName SalesJob

dsjob -jobinfo ProjectName JobName
Reset a job:
dsjob -run -mode RESET ProjectName JobName

Important fields:

- **UID** → user running the process
- **PID** → Process ID
- **PPID** → Parent Process ID
- **CMD** → command/process
- **ps command** is used to view **currently running processes**.

ps -ef | grep ds -> Searches for DataStage-related processes

o/p:

UID PID PPID CMD
dsadm 2456 1200 ...

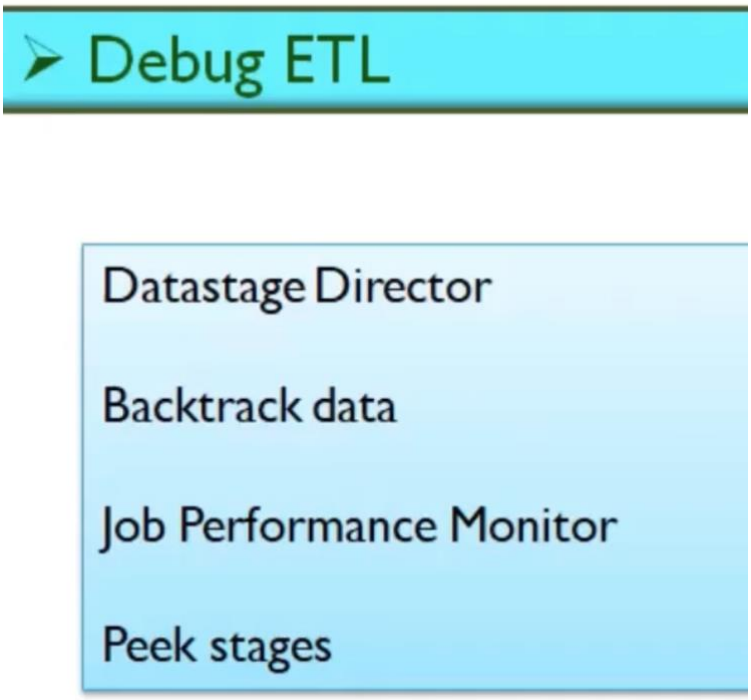
2. **DataStage can execute Unix commands**
   DataStage Job ↓ Execute Command ↓ Unix Shell Script ↓ Next DataStage Job

Command	Use
sed	Remove/change lines
cat	Concatenate files
grep	Search/filter records
wc -l	Count records
sort	Sort data

Command	Use
uniq	Find/remove duplicates
cut	Extract fields
awk	Complex file processing
head	Read first records
tail	Read last records
mv	Move/archive files
cp	Copy files
rm	Delete files
gzip	Compress files
test -f	Check file existence
test -s	Check file is non-empty

3. **shell script controlling a complete DataStage process**

Q . Debug Failed Datastage Job



Q .
Where do you see XML in DataStage?
In **DataStage Designer**, open the job and look for the XML-related stage, such as:

XML File
↓
XML Input / Hierarchical Data Stage
↓
Transformer
↓
Target
Ex:

<Stage Type="Transformer">
<Derivation> TOTAL_AMOUNT = QTY * PRICE </Derivation>
<Constraint> STATUS = 'ACTIVE' </Constraint>
</Stage>

SQL :
SELECT QTY, PRICE, QTY * PRICE AS TOTAL_AMOUNT
FROM SALES WHERE STATUS = 'ACTIVE';

Q. Datastage all Components

Category	Main DataStage Components	Main Use
Data Processing	Transformer	Derivation, validation, filtering, constraints, NULL handling
	Aggregator	SUM, COUNT, MIN, MAX, MEAN, grouping
	Sort	Sorting, duplicate handling, key-change column
	Remove Duplicates	Remove duplicate records based on keys
	Modify	Modify/drop/convert columns efficiently
	Copy	Copy one input stream to multiple outputs

Category	Main DataStage Components	Main Use
	Filter	Filter rows based on conditions
Join / Lookup	Lookup	Reference/master-data lookup
	Join	Inner, Left, Right, Full Outer joins
	Merge	Combine master/update datasets based on keys
Combining Data	Funnel	Combine multiple input streams into one
	Copy	One input → multiple outputs
File Stages	Sequential File	Read/write CSV, delimited, fixed-width files
	Data Set	DataStage-native parallel intermediate storage
	File Set	Store/read data across multiple files/partitions
	Lookup File Set	Optimized reference data for Lookup
	External Source/Target	Interact with external programs/files
Database / Connectors	Oracle Connector	Oracle read/write

Category	Main DataStage Components	Main Use
	DB2 Connector	DB2
	ODBC Connector	Generic DB connectivity
	JDBC Connector	JDBC-based connectivity
	Teradata Connector	Teradata
	Snowflake Connector	Snowflake, depending on installed environment/version
Hierarchical Data	Hierarchical Data Stage	XML/JSON hierarchical processing
	XML Input / XML Output	XML processing in traditional implementations
Debugging	Peek	View intermediate records
	Head	Select first N records, useful for testing
	Tail	Select last N records
	Sample	Take sample records
Development / Utility	Column Generator	Generate additional columns
	Row Generator	Generate test records

Category	Main DataStage Components	Main Use
Job Control	Job Activity	Execute another DataStage job
	Execute Command Activity	Run Unix/shell commands
	Routine Activity	Execute routines
	Nested Condition	Conditional execution
	Sequencer	AND/OR-style dependency control
	Notification Activity	Notifications where configured
Quality / Validation	Transformer	Custom validation/business rules
	Reject Link	Capture rejected/bad records
Parallel Processing	Partitioning	Hash, Round Robin, Same, Entire, Range, etc.
	Collecting	Combine partitioned streams when required

Mostly used components:

Transformer → Lookup → Join → Aggregator → Sort → Remove Duplicates → Funnel → Sequential File → Data Set → Database Connector → Peek → Sequence Job

Q . Implement SCD Type 2

Method	Stages/Approach	When used
1. SCD Stage	Slowly Changing Dimension Stage	Built-in/direct approach
2. Lookup + Transformer	Lookup → Transformer → Update/Insert	Most common custom approach
3. Change Capture	Change Capture → Transformer → Update/Insert	Compare old vs new datasets

Q. Peek Stage in DataStage
Peek Stage is mainly used for debugging/testing. It lets you view records flowing through a parallel job without changing the data.
Peek = See intermediate data → Debug → Job log.

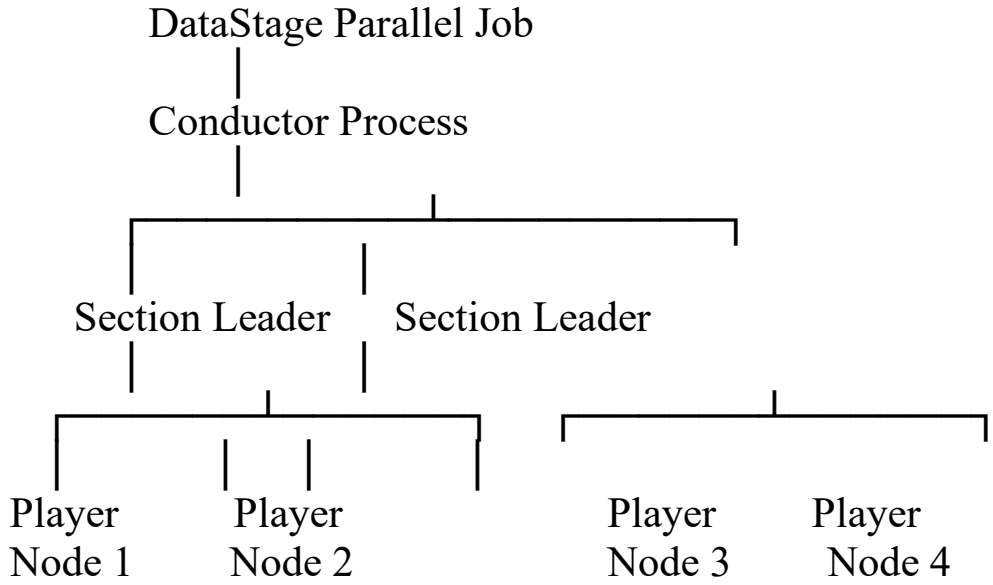
Q. Constraint vs Derivation in DataStage Transformer

Q. DataStage Functions

	Constraint	**Derivation**
Purpose	Decides **which rows pass**	Decides **what value goes into a column**
Works on	Rows	Columns
Similar to SQL	WHERE	SELECT expression
Result	Filter/routing	Transform/calculation

Q. **What are shared containers, and when should you use them?**
A Shared Container is a reusable group of stages and links that can be created once and used in multiple DataStage jobs.
Think of it like a reusable function/module.

Q. **DataStage Parallel Job Architecture**
A DataStage parallel job follows a shared-nothing parallel processing architecture. The job is divided across multiple processing nodes based on the configuration file.



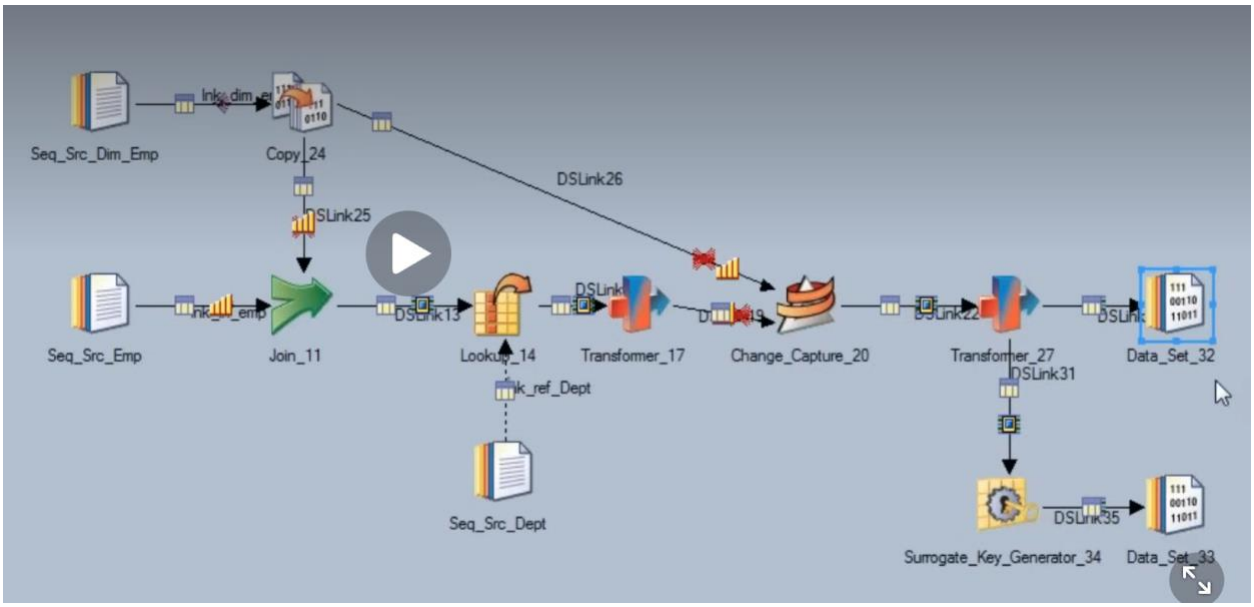
Q . SCD Type 1

Source ↓
Lookup ← Existing Target
↓ Transformer
↓ New? ——————→ INSERT
Changed? ——→ UPDATE
Same? ——————→ IGNORE
Lookup → Compare → Insert New → Update Changed → Ignore Same.
Ex :Before: 101 | John | Chicago
Source: 101 | John | Dallas
After: 101 | John | Dallas

Q. SCD Type 2

Approach	Performance	Flexibility	Best use
SCD Stage	Good	Medium	Standard SCD1/SCD2 requirements
Lookup + Transformer	Very good when lookup/reference fits the design	**High**	Custom business rules and selective change handling
Change Capture + Transformer	**Very good for large batch comparisons**	High	Comparing old vs new datasets/snapshots

CDC :



Change_Code = 1 → New employee
Change_Code = 3 → Changed employee
Change_Code = 0 → Ignore
Change_Code = 2 → Delete/expire depending on requirement

9. What are the functions of DataStage?

This platform has different types of functions, including -

Application	Function	Instance
Data Collection	Stage Function	Collecting data via APIs, web scraping (BeautifulSoup, Scrapy).
Data Storage	Database Function	SQL queries, database indexing, CRUD operations (MySQL, PostgreSQL).
Data Transformation	Type Conversion Functions	Casting data types (e.g., int(), str() in Python, CAST in SQL).
Data Cleaning	String Functions	String manipulation (SUBSTRING, REPLACE, TRIM in SQL/Python).
Data Processing	Additional Functions	Aggregations (SUM, AVG), filtering, sorting (Python, Pandas, Spark SQL).

Q. Load data into AWS – Object Storage Connector

Bucket = sales-data
Object/Path = output/sales.csv
Format = CSV
Credentials = configured AWS credentials

More stages:
Google Cloud Storage Connector
Google BigQuery Connector